Spark

Introduction to Data Mining

Seoul National University

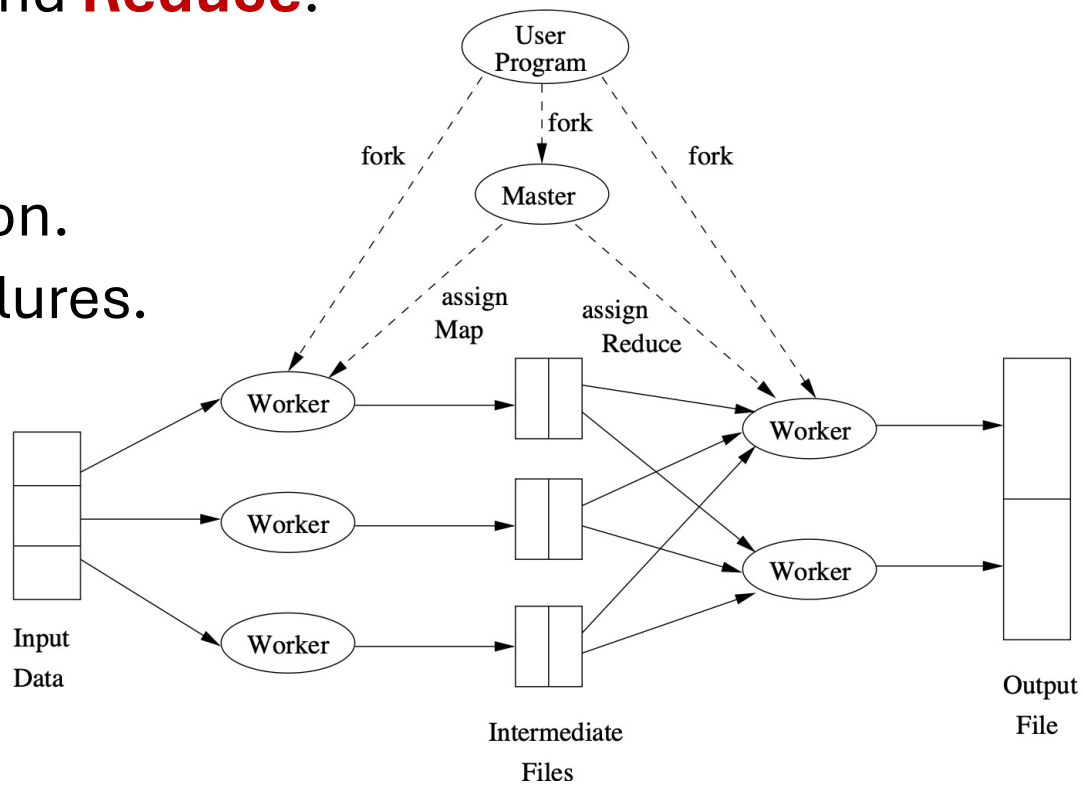
Spring 2026

Announcements

- **Communications:**
  - We will use eTL for all course communications (no other apps).
- **Assignments:**
  - Please check the announcement for the computing environment.
  - HW0 will be posted today. No submissions required.

Recap: MapReduce

- **MapReduce** is suitable for large files rarely updated in place.
  - User implements two functions: **Map** and **Reduce**.
    - Map: $(k, v) \rightarrow \{(k_i, v_i)\}_i$
    - Reduce: $(k, [v_1, \cdots, v_n]) \rightarrow (k, v)$
  - System runs the group-by-keys operation.
  - System automatically handles node failures.



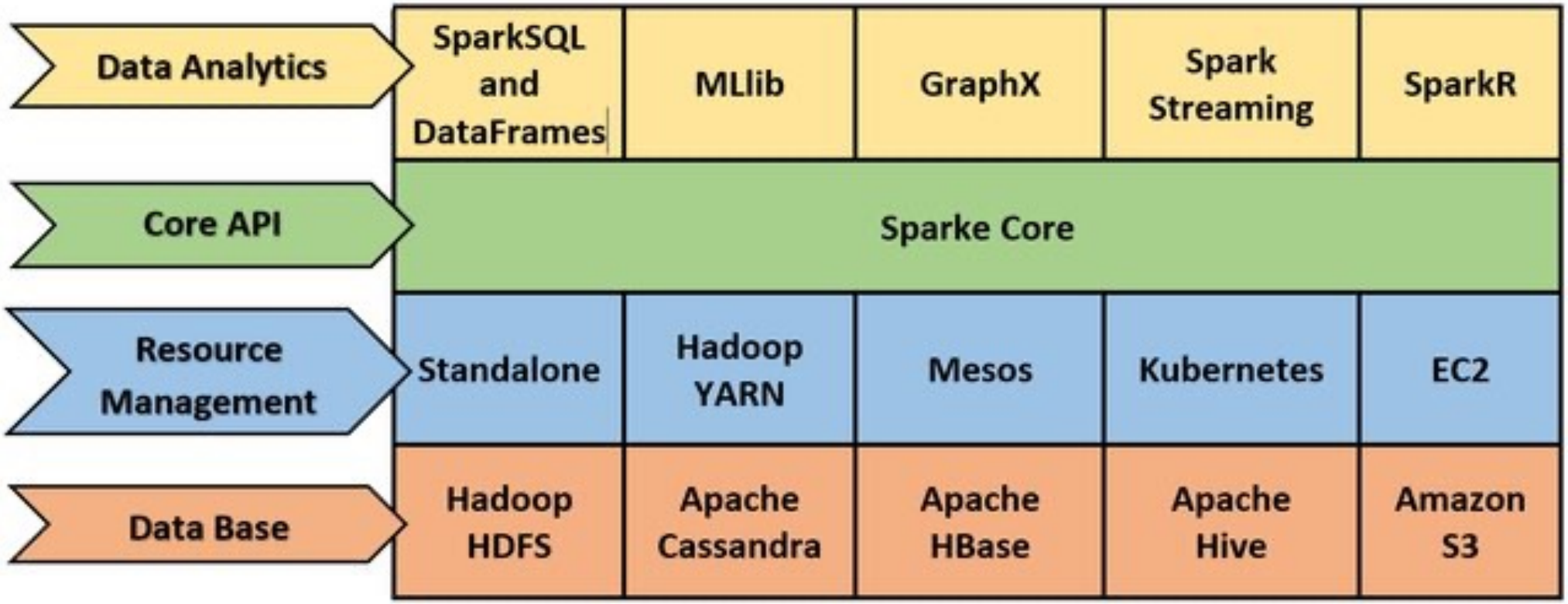
Outline

1. **<u>Spark: Overview</u>**
2. Spark: Operations
3. Spark: Implementation

Hadoop

- **Hadoop** is an open-source framework for distributed processing.
  - MapReduce is tightly coupled with Hadoop.
- Consists of three key components:
  - **HDFS:** Stores large datasets across clusters with replication.
  - **YARN:** Manages resources and job scheduling across nodes.
  - **Processing engines:** Run computations (e.g., MapReduce and Spark).
- In modern Hadoop, the master controller mostly means YARN.

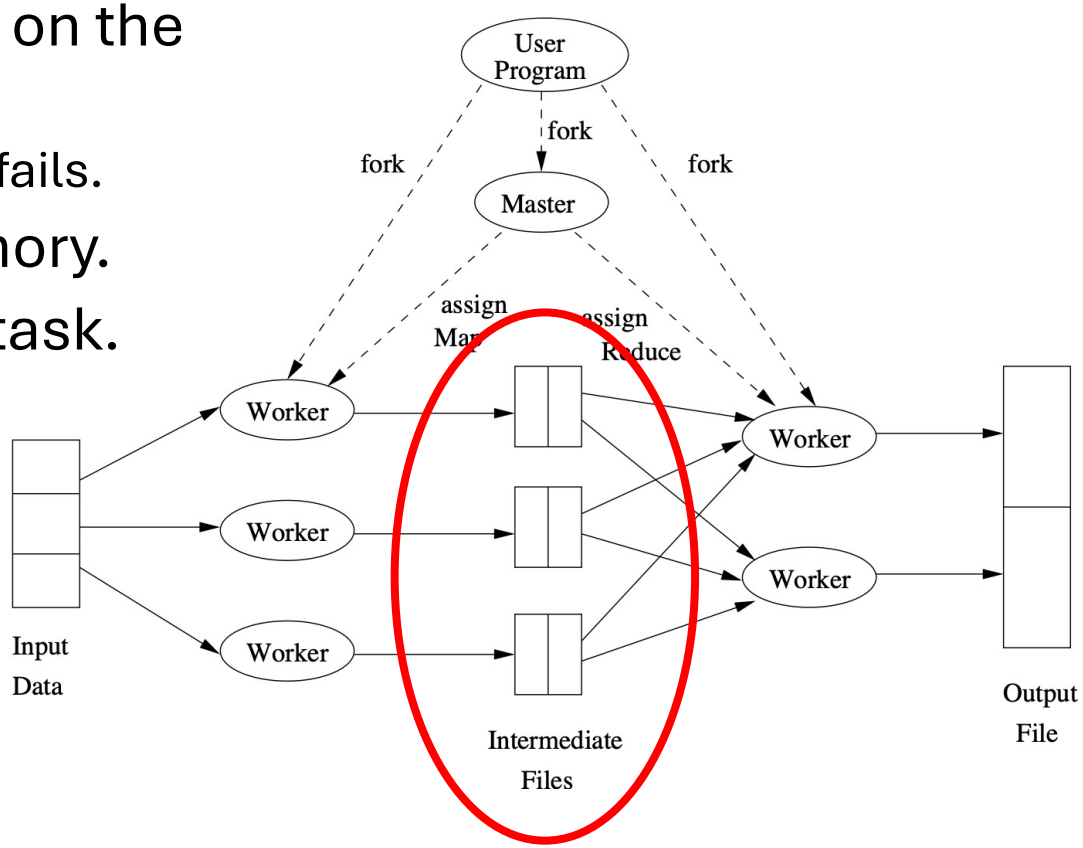
Data Analytics Software Stack



Source: Maleki et al. (2019)

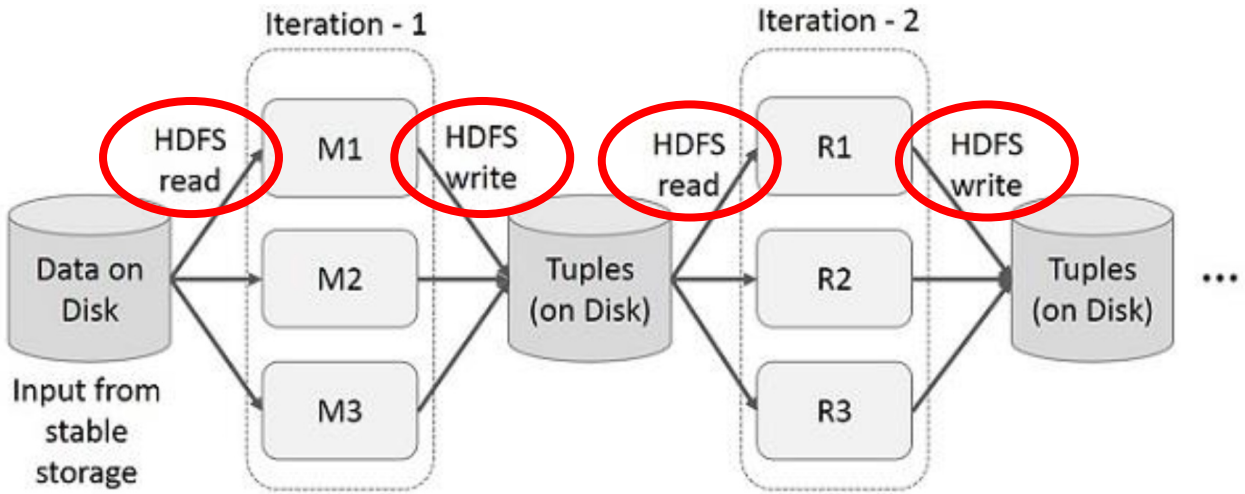
Limitations of MapReduce (1/2)

- MapReduce stores intermediate files on disk.
  - The output of each Map task is stored on the local file system of the worker.
    - The files are not accessible if the worker fails.
  - **Pros:** Can be run with little main memory.
  - **Cons:** Disk overhead after each Map task.



Limitations of MapReduce (2/2)

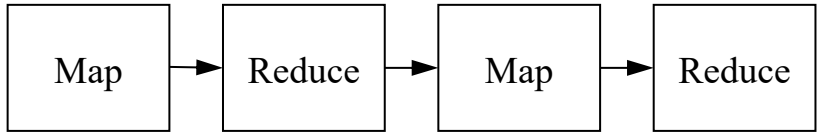
- MapReduce incurs substantial overheads for a complex job.
  - Due to data replication and disk I/O.
- Many algorithms are not easily described as MapReduce.

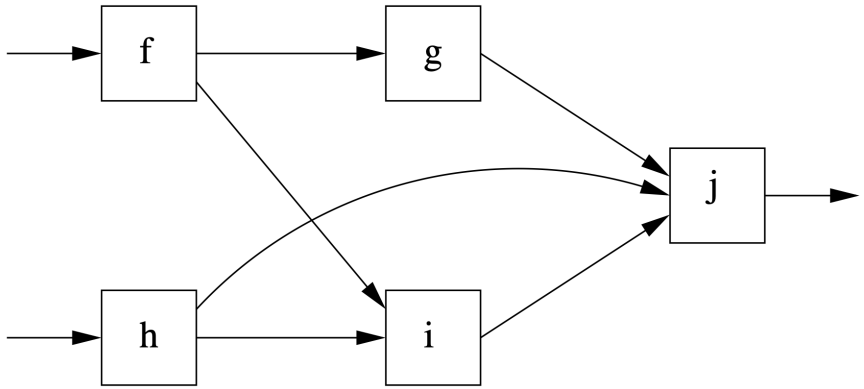


Source: Stanford CS246 (2022)

Workflow System

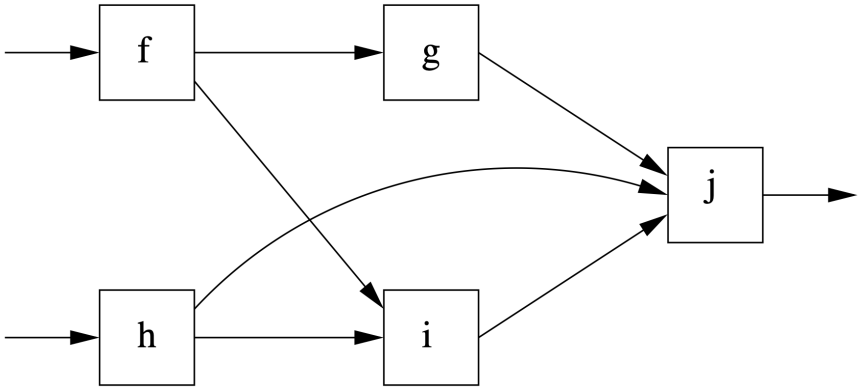
- MapReduce creates a **chain** of tasks (Map–Reduce–Map–).
- Let's extend it into a **directed acyclic graph (DAG)** of tasks.
  - Allow functions that can take more than one input.
- Master controller is responsible for dividing work among tasks.





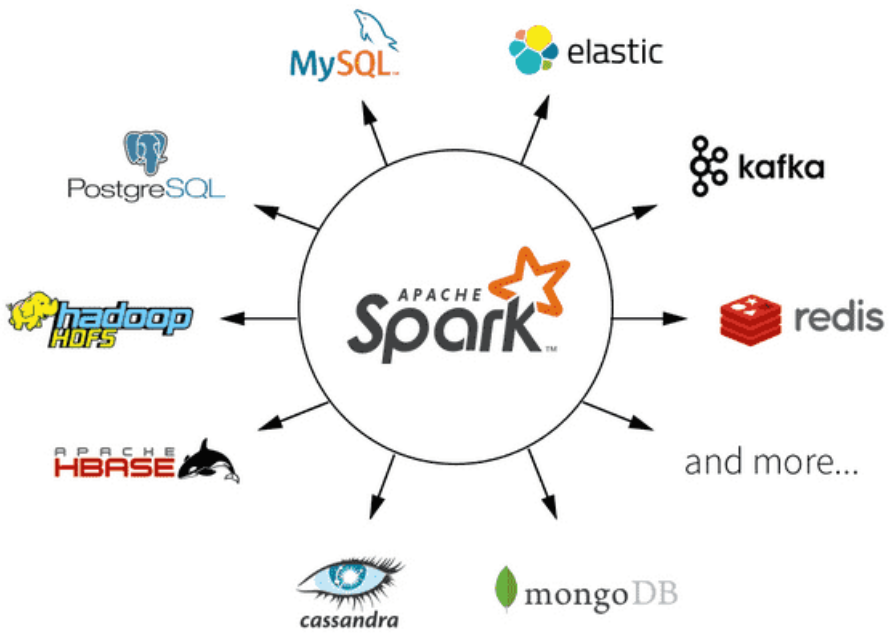
Blocking Property

- **Blocking property** allows a fault-resilient model.
  - Operation can run only after all its predecessors are completed.
  - If a task fails, we know that any of its successors didn't start.
- Master controller can restart failed tasks at another node.
  - If g fails, then we re-run f and then g following the graph.



Spark: Most Popular Workflow System

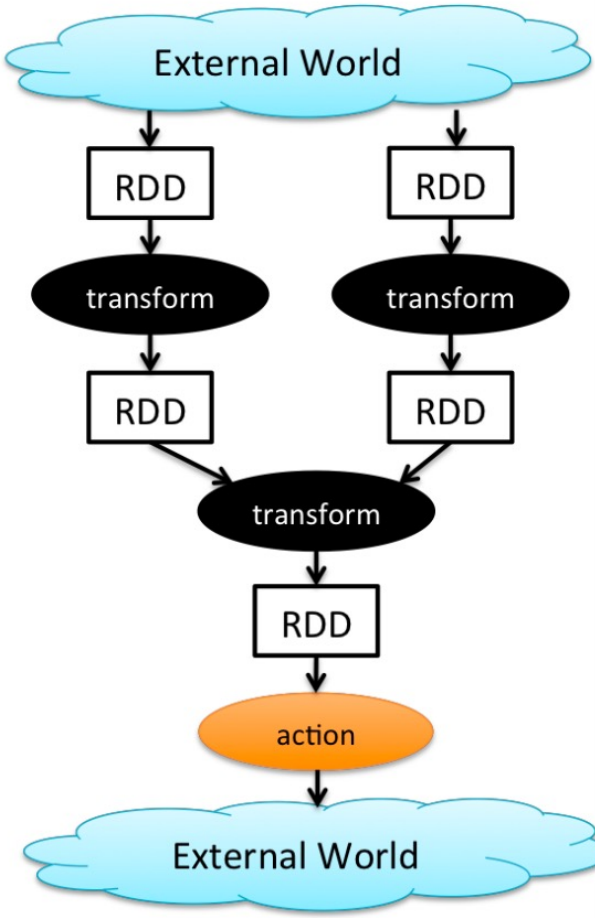
- **Spark** is a flexible workflow system not limited to MapReduce.
  - Developed by UCB and Databricks, now maintained by Apache.
- **Additions to MapReduce:**
  - Fast data sharing
    - Avoid saving intermediate results to disk.
    - Cache data for repetitive queries (e.g. for ML).
  - General execution graphs (DAGs)
  - Richer functions than Map and Reduce
- Compatible with Hadoop (e.g., HDFS).



Source: DataBricks

Resilient Distributed Dataset (RDD)

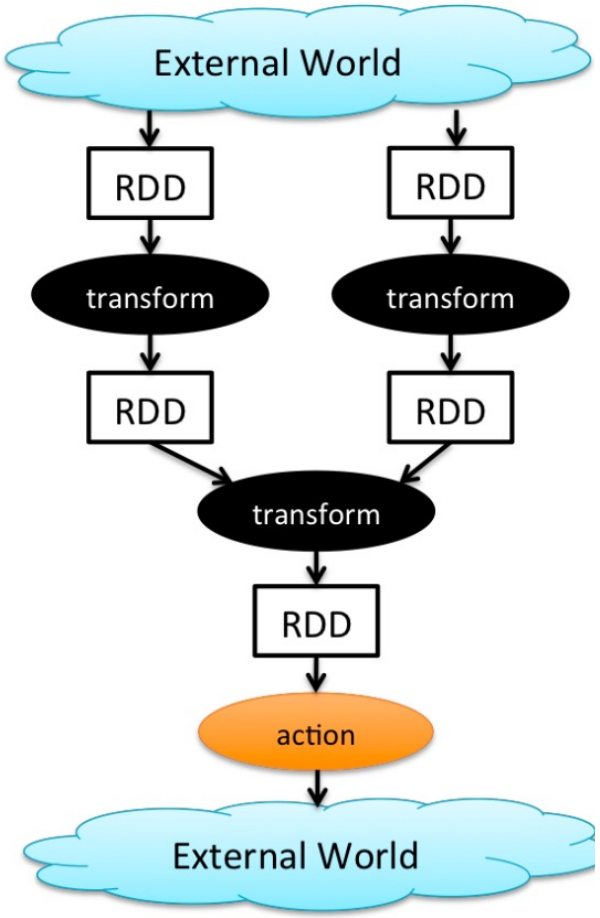
- **RDD:** Key idea / central data abstraction of Spark.
  - Partitioned collection of records of **one type**.
    - Generalizes key-value pairs in MapReduce.
  - Spread across the cluster (i.e., chunks) and read-only.
  - Can be **cached** in main memory.
  - Created from DFS or by transforming other RDDs.
  - Best if the same operation is applied to all elements.



Source: Stanford CS246 (2022)

Spark Program

- **Transformations** build RDDs from other RDDs.
  - Through deterministic, parallelizable operations.
  - Include *map*, *filter*, *join*, *union*, *distinct*, etc.
  - **Lazy evaluation:** Nothing is computed until an action.
- **Actions** are used to return value or export data.
  - Include *count*, *collect*, *reduce*, *save*, etc.
  - Applied to RDDs, force calculations and return values.



Source: Stanford CS246 (2022)

Outline

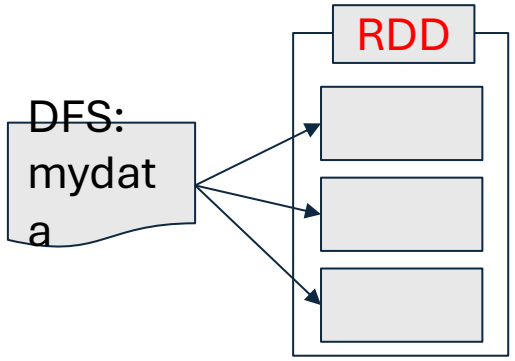
1. Spark: Overview
2. **<u>Spark: Operations</u>**
3. Spark: Implementation

Example: Average Word Length by Letter

- **Problem:**
  - Given a large text file (stored in the DFS),
  - For each letter $\pi \in \{a, \cdots, z\}$,
  - What is the average length of words that start with π?
- **How to do it with MapReduce:**
  - Map: $(\text{line index}, \text{line}) \rightarrow \{(w[0], \text{len}(w)) \mid w \in \text{line}\}$
  - Reduce: $(\text{word}, \text{lengths}) \rightarrow (\text{word}, \text{mean}(\text{lengths}))$

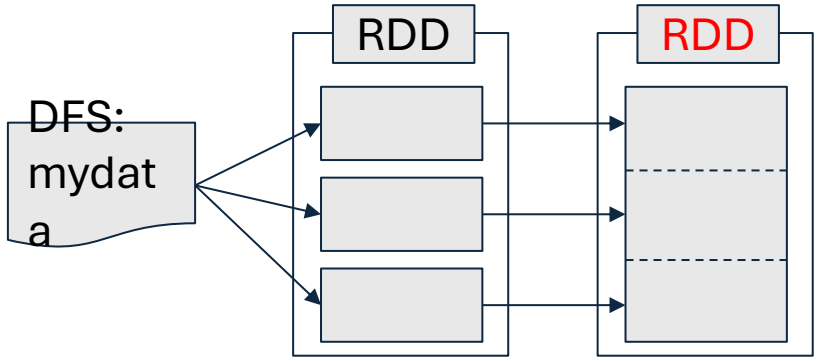
Example: Average Word Length by Letter

```
> avglens = sc.textFile(file)
```



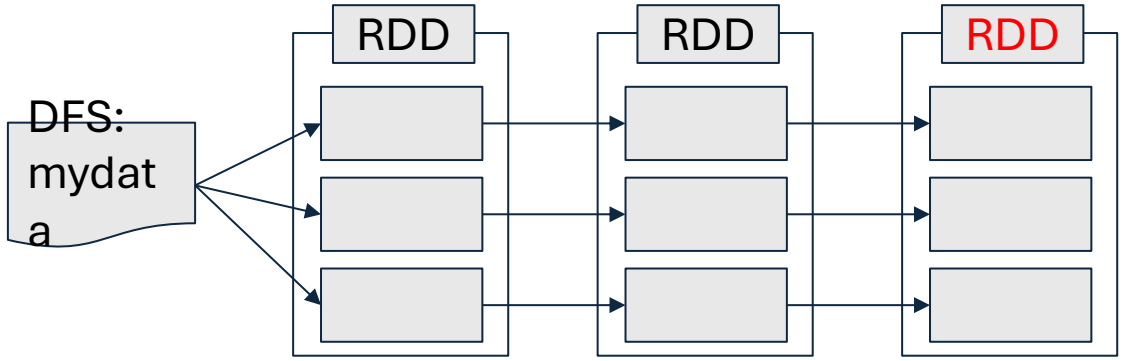
Example: Average Word Length by Letter

```
> avglens = sc.textFile(file) \
   .flatMap(lambda line: line.split())
```



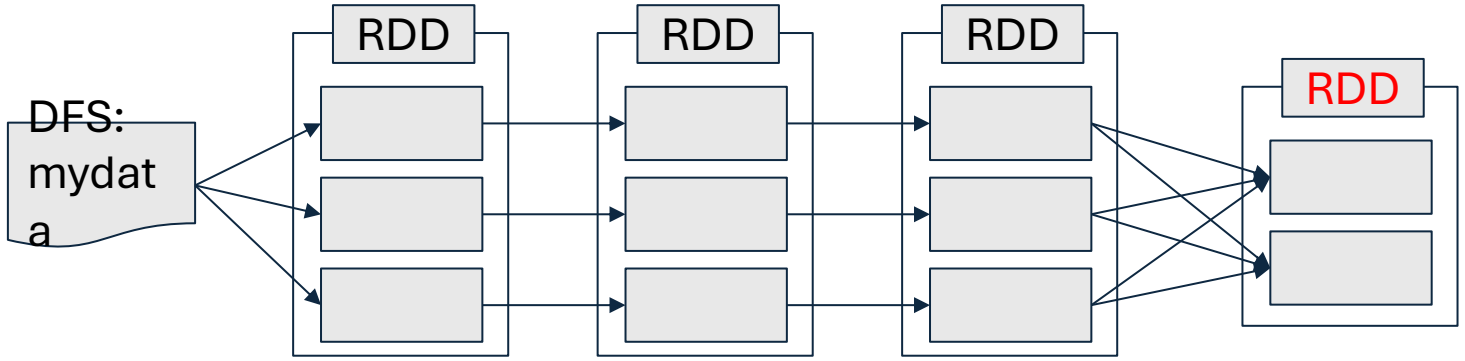
Example: Average Word Length by Letter

```
> avglens = sc.textFile(file) \
    .flatMap(lambda line: line.split()) \
    .map(lambda word: (word[0], len(word)))
```



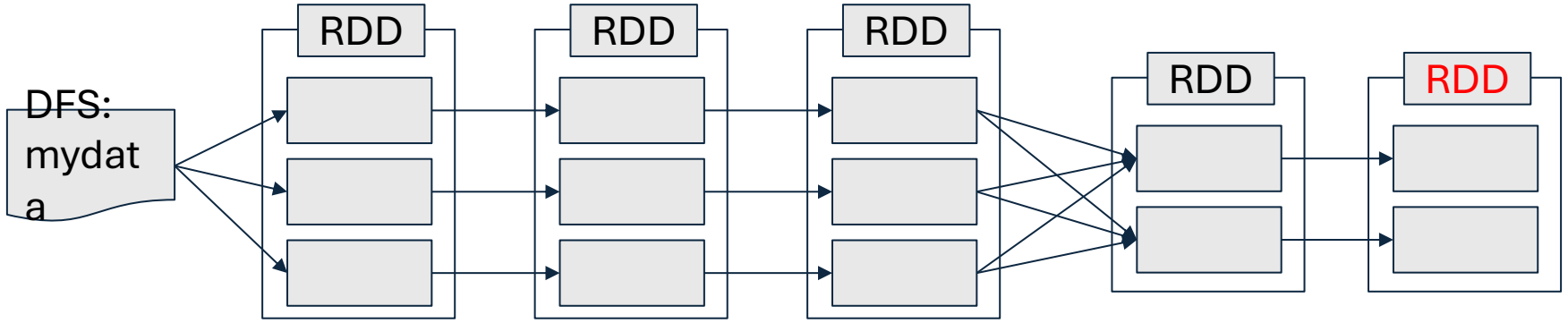
Example: Average Word Length by Letter

```
> avglens = sc.textFile(file) \
    .flatMap(lambda line: line.split()) \
    .map(lambda word: (word[0], len(word))) \
    .groupByKey()
```



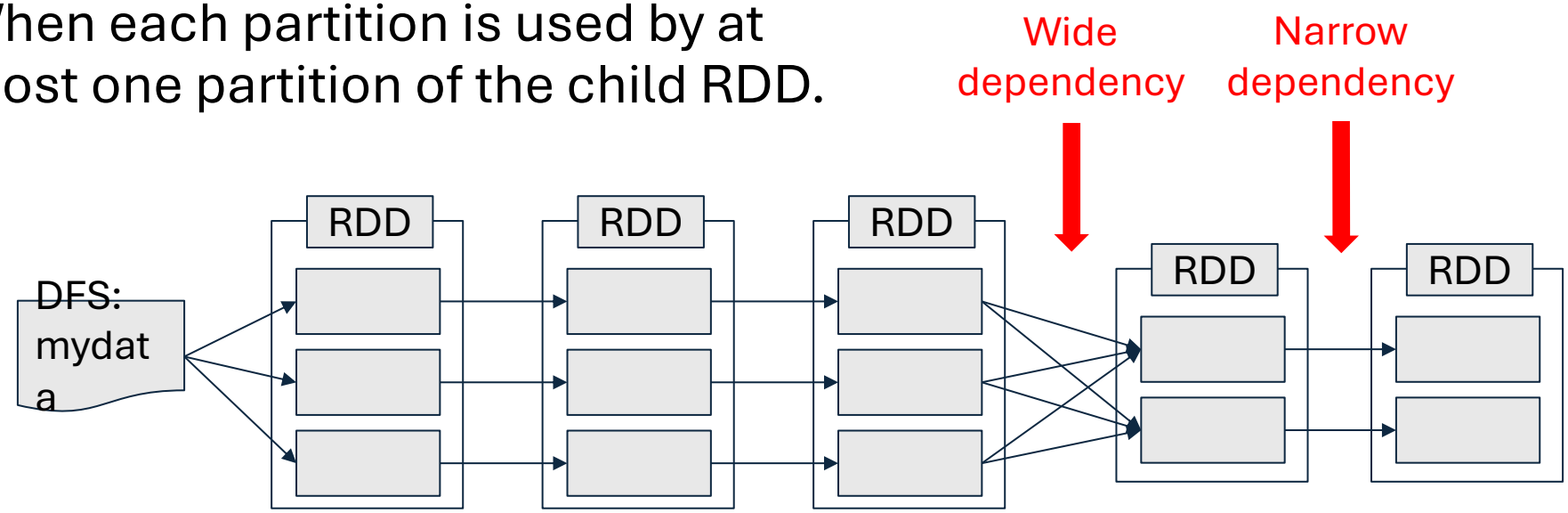
Example: Average Word Length by Letter

```
> avglens = sc.textFile(file) \
    .flatMap(lambda line: line.split()) \
    .map(lambda word: (word[0], len(word))) \
    .groupByKey() \
    .map(lambda (k, values): \
      (k, sum(values) / len(values)))
```



Example: Average Word Length by Letter

- **Wide dependency** (slow; shuffling)
  - When each partition is depended on by multiple child partitions.
- **Narrow dependency** (fast; pipelining)
  - When each partition is used by at most one partition of the child RDD.



FlatMap

- Transformation that applies a function to every element.
- Same as MapReduce's Map, but has no restriction on the type.
  - Maps **any object** into a list of **0 or more objects**.
- All the lists are *flattened* into a single RDD of output objects.

```
> avglens = sc.textFile(file) \
    .flatMap(lambda line: line.split()) \
    .map(lambda word: (word[0], len(word))) \
    .groupByKey() \
    .map(lambda (k, values): \
      (k, sum(values) / len(values)))
```

Map

- Stricter version of the FlatMap transformation.
- Forces to create **exactly one object** from each input element.
  - No flattening operation is applied.
  - System has better control for the resource management.

```
> avglens = sc.textFile(file) \
   .flatMap(lambda line: line.split()) \
   .map(lambda word: (word[0], len(word))) \
   .groupByKey() \
   .map(lambda (k, values): \
     (k, sum(values) / len(values)))
```

Filter

- Takes a predicate that applies to the RDD object type.
- Returns elements that satisfy the predicate.
- Better be applied as early as possible to reduce the RDD's size.

```
> avglens = sc.textFile(file) \
   .flatMap(lambda line: line.split()) \
   .filter(lambda word: word not in stoplist) \
   .map(lambda word: (word[0], len(word))) \
   .groupByKey() \
   .map(lambda (k, values): \
     (k, sum(values) / len(values)))
```

Reduce

- Action (not transformation) that returns a value instead of an RDD.
- Takes parameter that is a function of type $(V, V) \Rightarrow V$.
  - The function is repeatedly applied on pairs of elements in any order.
  - Each pair reduces to one element, and we are left with only one.
  - The function should be **associative** and **commutative** (e.g., addition).

```
> avglens = sc.textFile(file) \
    .flatMap(lambda line: line.split()) \
    .map(lambda word: len(word)) \
    .reduce(lambda a, b: a + b)
```

Join

- Transformation that merges two RDDs of key-value pairs.
  - They should have the same key types.
  - For each pair (k, x) and (k, y), it creates $(k, (x, y))$.
- Creates a **wide dependency**.

```
> x = sc.parallelize([("a", 1), ("b", 4)])
> y = sc.parallelize([("a", 2), ("a", 3)])
> x.join(y).collect()
[('a', (1, 2)), ('a', (1, 3))]
```

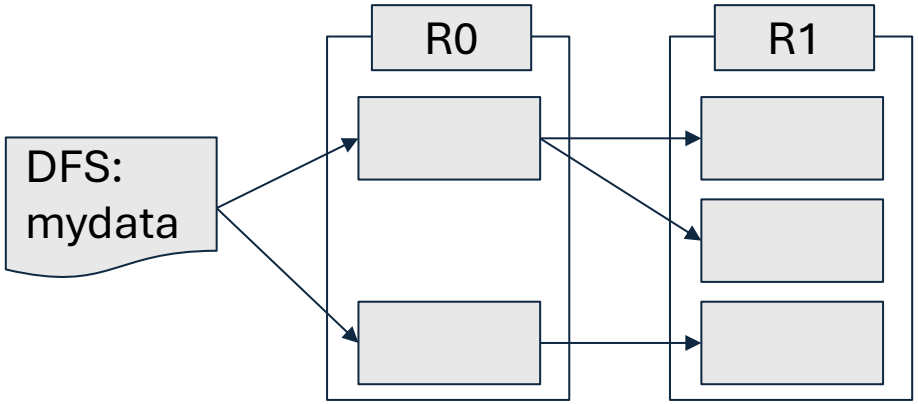
GroupByKey

- Transformation that is the same as MapReduce's group-by-keys.
  - Takes an RDD of key-value pairs.
  - Produces an RDD of key-"value list" pairs.
- Creates a **wide dependency**.

```
> avglens = sc.textFile(file) \
    .flatMap(lambda line: line.split()) \
    .map(lambda word: (word[0], len(word))) \
    .groupByKey() \
    .map(lambda (k, values): \
      (k, sum(values) / len(values)))
```

Wide vs. Narrow Dependencies

- **Q1:** Can FlatMap create multiple partitions from one?
  - **A:** Yes, it is a one-to-many function.
- **Q2:** Then, does it have a **wide dependency**?
  - **A:** No, each partition in R1 still comes from only one partition.



Pop Quiz

- There are many other transformations and actions on Spark.
- E.g., `reduceByKey(func)` is like `groupByKey()`, but also applies a reduce function `func` of the form $(V, V) \Rightarrow V$ on the values.
  - Better than `groupByKey() + reduce()`.
- **Problem:** Implement word count using `reduceByKey()`.

Outline

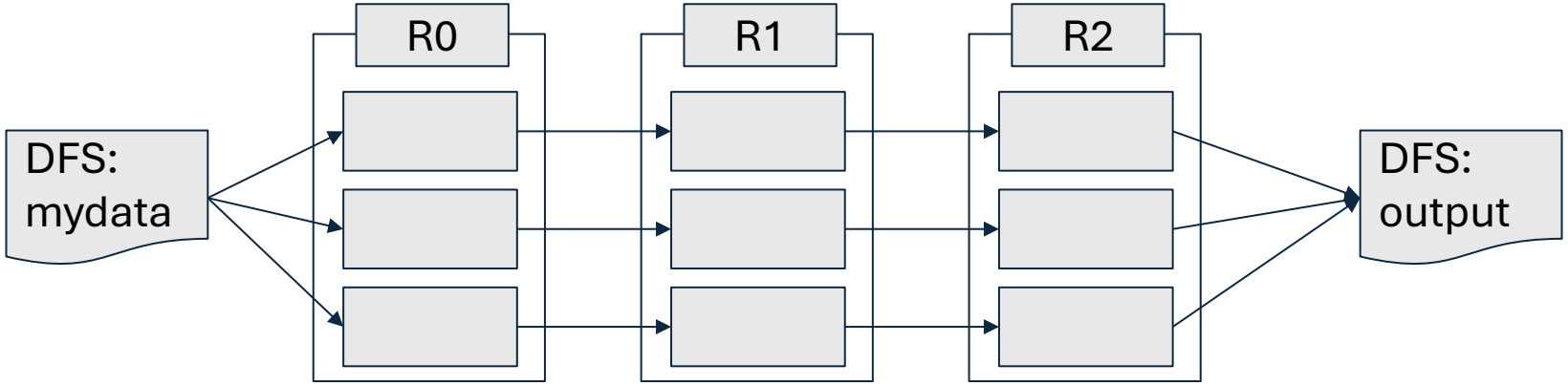
1. Spark: Overview
2. Spark: Operations
3. **<u>Spark: Implementation</u>**

Spark Implementation

- **Similarities with MapReduce:**
  - RDD is divided into chunks, which are given to different nodes.
  - Transformation on RDDs can be performed in parallel on chunks.
- **Two key improvements over MapReduce:**
  - Lazy evaluation of RDDs (for efficiency)
  - Lineage for RDDs (for fault-resilience)

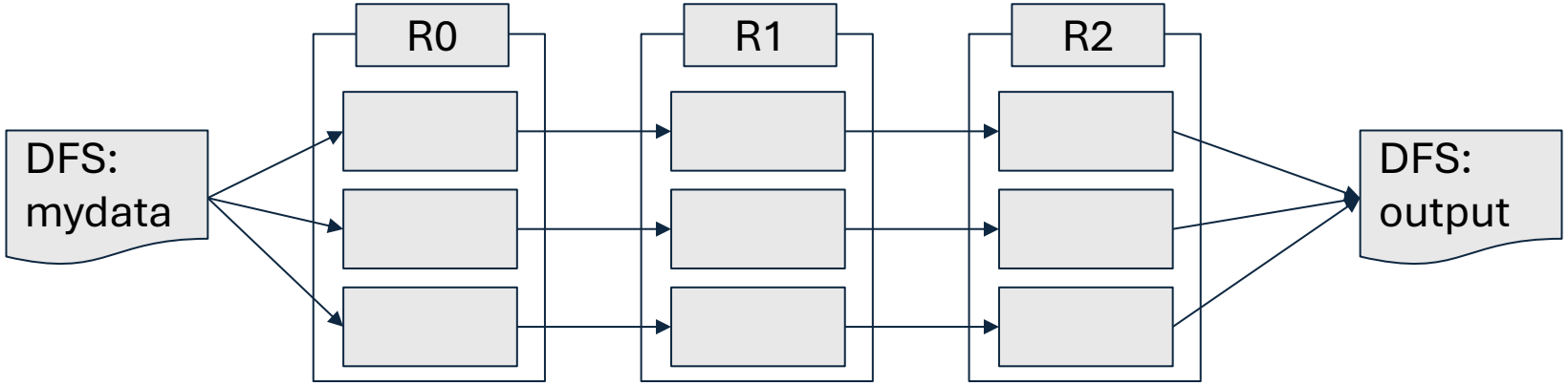
Lazy Evaluation

- Spark does not apply transformations until it is required to do so.
  - E.g., storing an RDD to file system or returning a result to application.
- Thanks to **lazy evaluation**, Spark can optimize execution plans.
  - Can combine transformations, minimizing the number of passes.
  - Can pass the result of a transformation **in memory** without using disk.



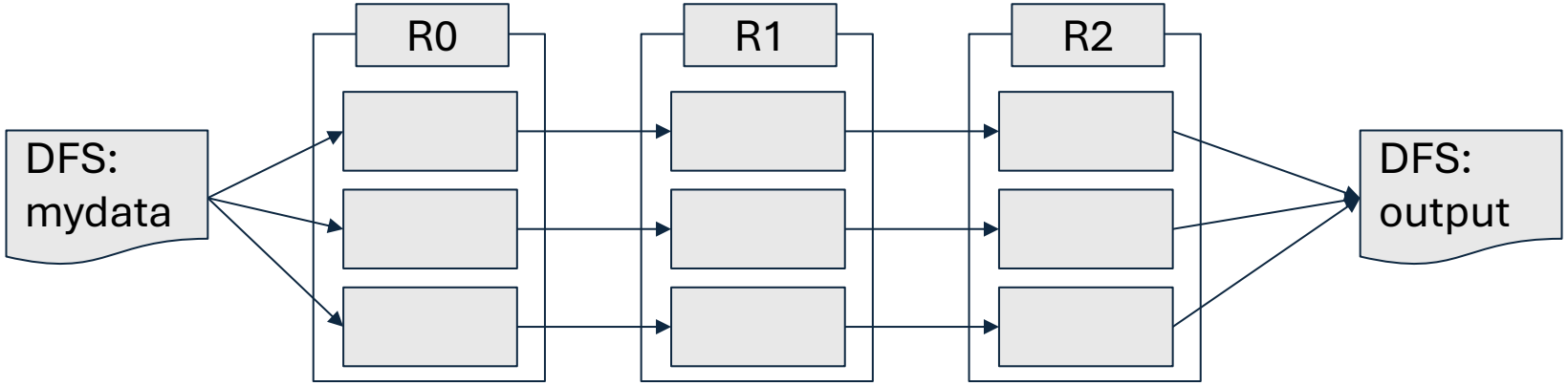
Lazy Evaluation Example

- Count words that are not stop words in a document.
  - Apply `Flatmap` to input RDD R0 to create $(w, 1)$ pairs in R1.
  - Apply `Filter` to each chunk in R1 to produce R2.
  - Only if R2 is stored in DFS (by `Reduce` action) then R1 and R2 are actually produced and subsequently dropped after being used.



Resilience of RDDs

- Spark records the **lineage** of every RDD for fault-resilience.
  - Use it to re-create the RDD in case of the node failure.
  - If R2 is lost, reconstruct from R1.
  - If R1 is lost, reconstruct from R0.
  - If R0 is lost, reconstruct from the file system.



Spark Programming Guide and Paper

- To learn more about writing Spark applications, please read:
  - https://spark.apache.org/docs/latest/rdd-programming-guide.html
- (Optional) More technical details of Spark are in this paper:
  - https://www.usenix.org/system/files/conference/nsdi12/nsdi12-final138.pdf

MapReduce vs. Spark

	MapReduce	Spark
Speed	Faster than traditional systems	Much faster (100x) than MapReduce
Written in	Java	Scala
Data Processing	Batch processing	Batch / real-time / iterative / interactive / graph
Ease of Use	Complex and lengthy	Compact and easier than MapReduce
Caching	No caching	Caches data in memory, enhancing system performance

When to Use What

- **MapReduce:**
  - Linear processing of large datasets.
  - Simple tasks easily written as Map and Reduce.
  - Offline, large-scale batch jobs
- **Spark:**
  - Fast and interactive data processing.
  - Iterative and real-time jobs like machine learning.
  - Especially when each node has sufficient memory.

Summary

1. Spark: Overview
   - Resilient distributed datasets (RDD)
2. Spark: Operations
   - Map, FlatMap, Filter, Reduce, Join, etc.
3. Spark: Implementation
   - Lazy evaluation
   - Lineage of RDDs